

Modernizing Load Frame Software: From Text Files to Graphical User Interface

Tyler Black — Fort Lewis College
Mentor: Dr. Kelly Nygren
Cornell High Energy Synchrotron Source

Summer 2026

1 Purpose

The objective of this work is to design and implement a graphical user interface that replaces the existing text-based interface by researching user needs and preferences as well as system requirements and design considerations.

Rotation and Axial Motion System

Standard commercial load frames impose mechanical and geometric constraints on synchrotron experiments. The Rotation and Axial Motion System IV (RAMS IV) load frame solves these limitations by introducing dynamic tension, torsion, 360° rotation, and wide-angle beam clearance.

Existing RAMS IV User Interface

The existing RAMS IV interface is a Python codebase centered around editing multiple text files and navigating a command-line interface (CLI). The various text files in the codebase can be classified into two main groups—configuration files and experiment files.

A typical synchrotron experiment may require four JSON configuration files to be modified. It is common for a user to copy configuration files or clone a repository containing the configuration file so they have a baseline configuration that can be edited. Meanwhile, experiment files generally involve more interaction and the number of files can vary greatly between experiments. Experiment files are mostly space-delimited TXT arrays that use either custom key-value syntax or a spreadsheet-like numerical input. Editing raw text files not only carries a high barrier to entry, but it also opens up the possibility of typos or syntax errors as well as inconsistent methods and tools for editing the experiment files.

Experiment and configuration files must also be correctly placed in the file system according to how the user configured the paths in the main configuration file. This is most quickly done using the command line, but users can opt to use a graphical file system, too. Before the experiment can begin, the load frame hardware must be initialized by executing a Python script to launch a separate CLI program, which allows the user to directly control the RAMS hardware. In a separate terminal, the user then begins the experiment by executing the main Python script.

2 Capabilities & Results

The primary goal is a graphical user interface (GUI) that completely abstracts away the underlying Python codebase and associated text files. Currently, the *Configuration* and *Sequence Builder* sections of the project are in a usable state, and contain enough functionality to eliminate the need for creating any text files.

Configuration

The Configuration feature consists of three tabs for configuring RAMS along with creating necessary X-ray scan *profiles* and Digital Image Correlation (DIC) *profiles*. Within the three tabs, users have everything needed to fully configure an experiment. A version-controlled settings menu, accessible via the left sidebar, is also part of the Configuration feature. Updating any settings is a global operation, so there are additional warnings and pop-ups to deter an inexperienced user from editing the settings.

Sequence Builder

The Sequence Builder is an intuitive drag-and-drop mechanical test editor that supports several features such as grouping subsequences of commands, looping/repeating groups, and writing custom commands. The Sequence Builder replaces one of the most time-consuming parts of the existing text-based interface and gives the user more features and control for building a mechanical test.

3 Architecture

The TypeScript React web application is styled with Tailwind CSS, and it is programmed using a feature-driven architecture. The GUI is meant to act as a frontend to either a gateway application programming interface (API) or directly to the existing Python interface.

Payload Schemas & State Management

All payloads are formatted as JSON objects and are intended to be predictable and reusable. The three core payloads—settings, configuration, and test sequence—follow a Zod schema which verifies the user’s parameters before a request can be made. The application maintains three core payloads across two drafts—*Configuration* and *Mechanical Test*—which are globally persisted in local storage using Zustand. Any time a user modifies a field, the draft is compared against the previously saved draft, dictating whether the user has unsaved changes or not.

4 Project Status

The graphical user interface has been successfully developed as a functional prototype and is ready for integration and testing. Known issues, complete documentation, and a user guide are available in the project’s GitHub repository. Further development of this project is expected, and the project will be handed off to the current CHESS developers with an expected deployment this fall.